

ロジスティック回帰

分類の `fit()` を開ける

rkskmt

1

ここまでの道具

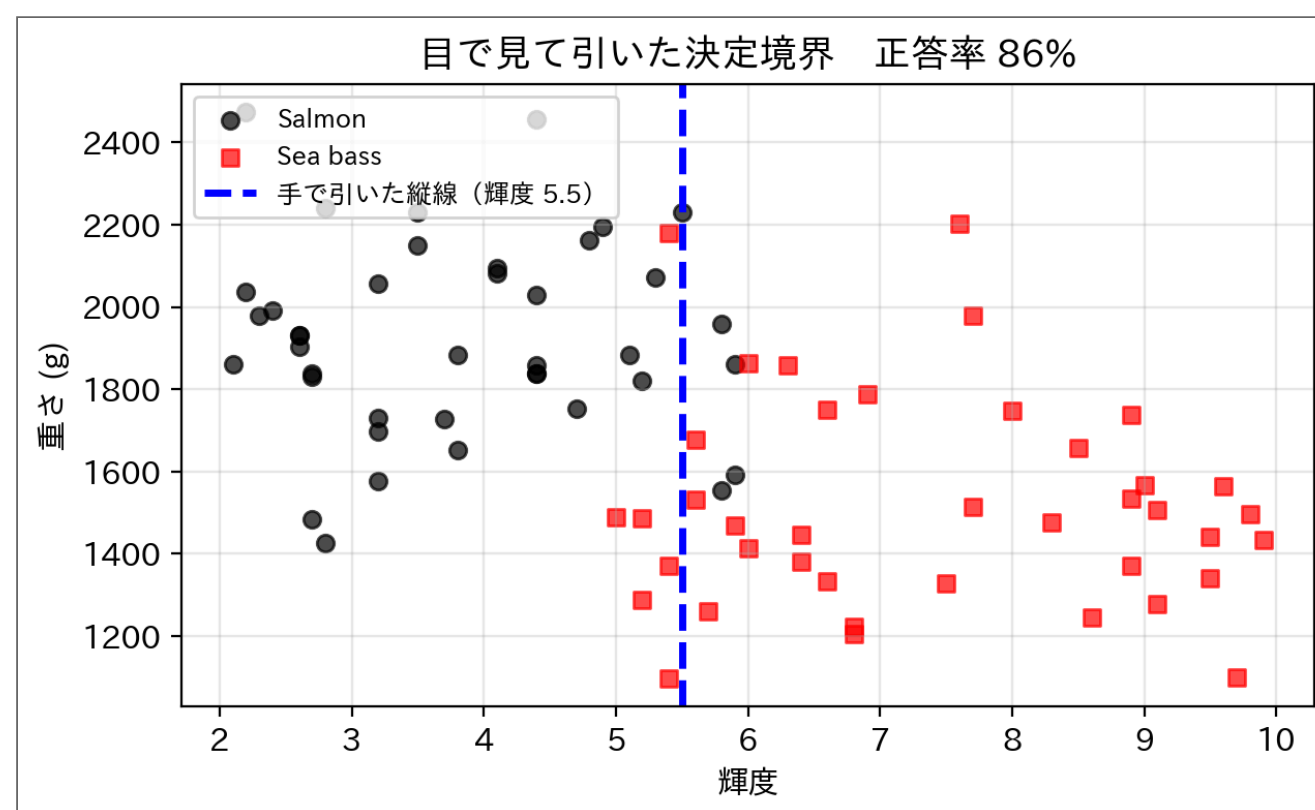
- **勾配降下法**：傾きの逆方向に少しずつ進んで最小値を探す（最適化の回）
- **回帰の `fit()` は自作済み**：体長→重さの直線を勾配降下で当て、sklearn と一致した（前回）
- 分類はどうか — 決定境界の回では、直線を **目で見**て引いた

① 今回の問い

「斜めの直線で分けられそう」と言いながら、引けたのは縦線だけだった。 **斜めの最適な直線**を、機械に fit させられるか？

2

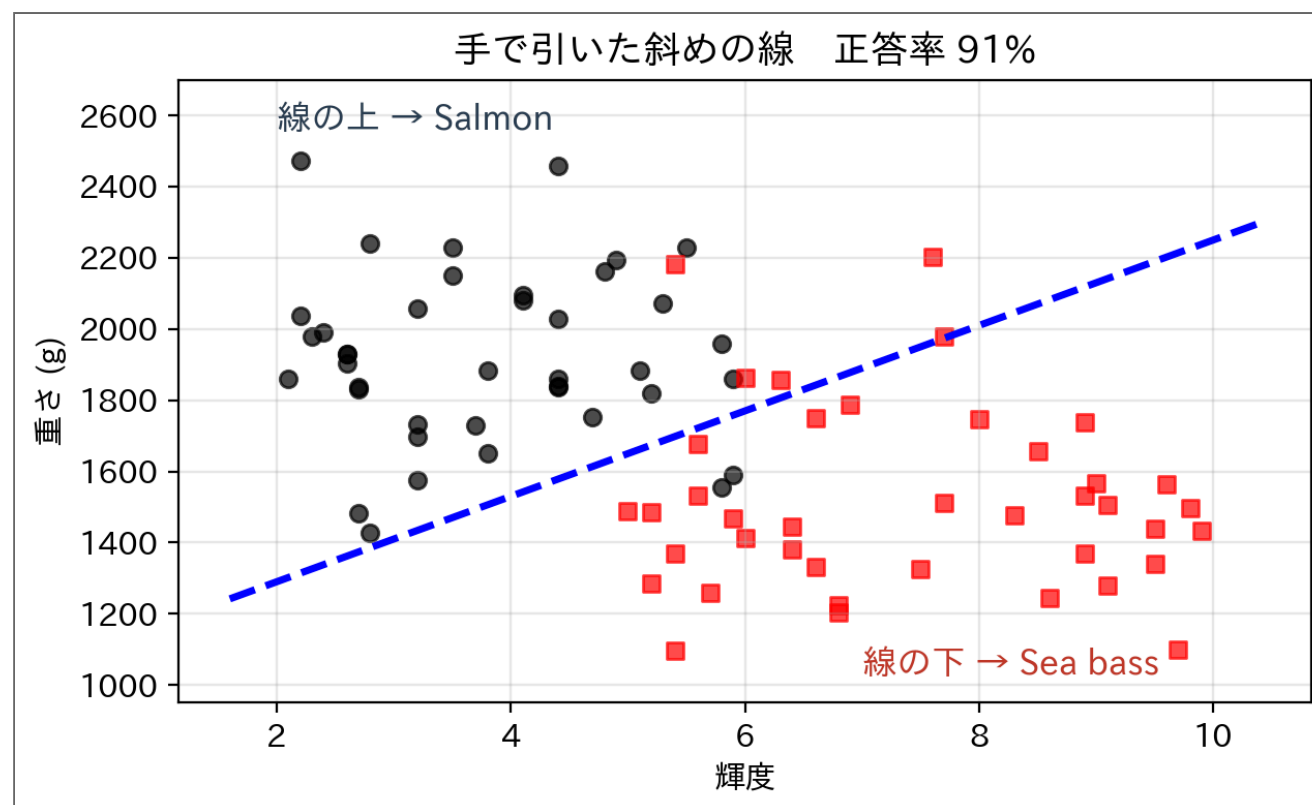
あのときの縦線



- 縦線（輝度 5.5）の正答率は **86%**（決定境界の回で手計算した）
- 目で見ると、**右上がりの斜め**ならもっと分けられそう

3

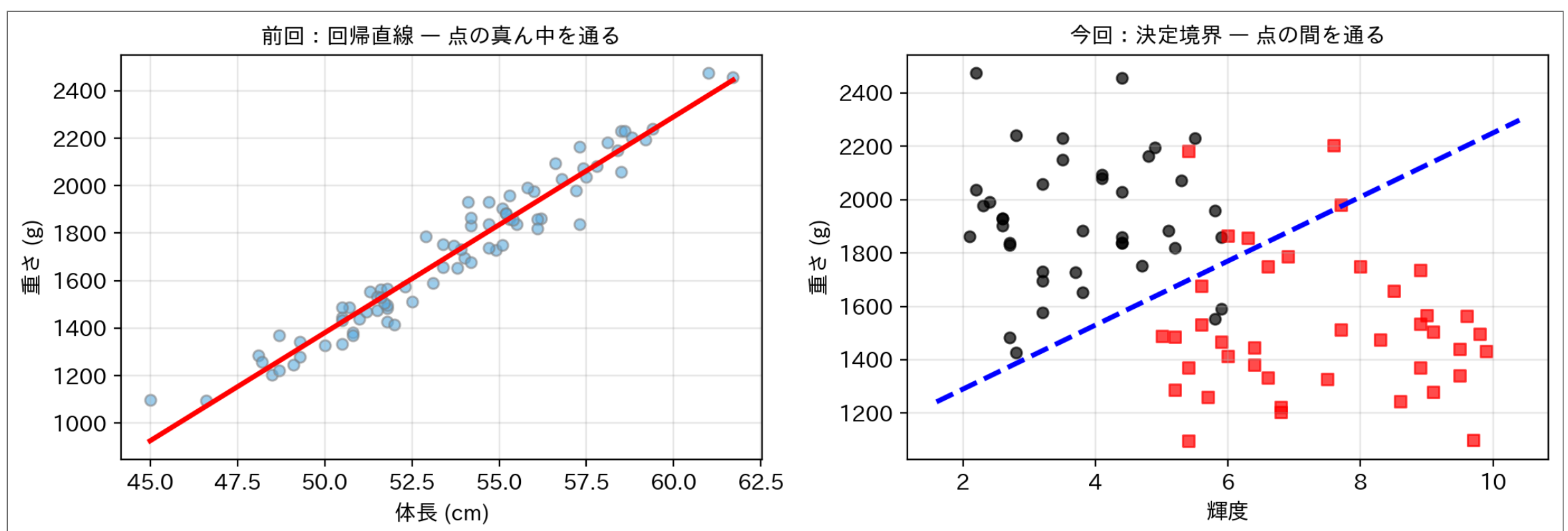
この線を求めたい



- 例えばこの線：正答率 **91%**（縦線は 86% だった）
- この線さえあれば、判別は「線の上か下か」だけ — これが**決定境界**
- ただし「斜め」は無限にある。最適の保証もない — **機械に求めさせたい**

4

この線は、回帰直線ではない



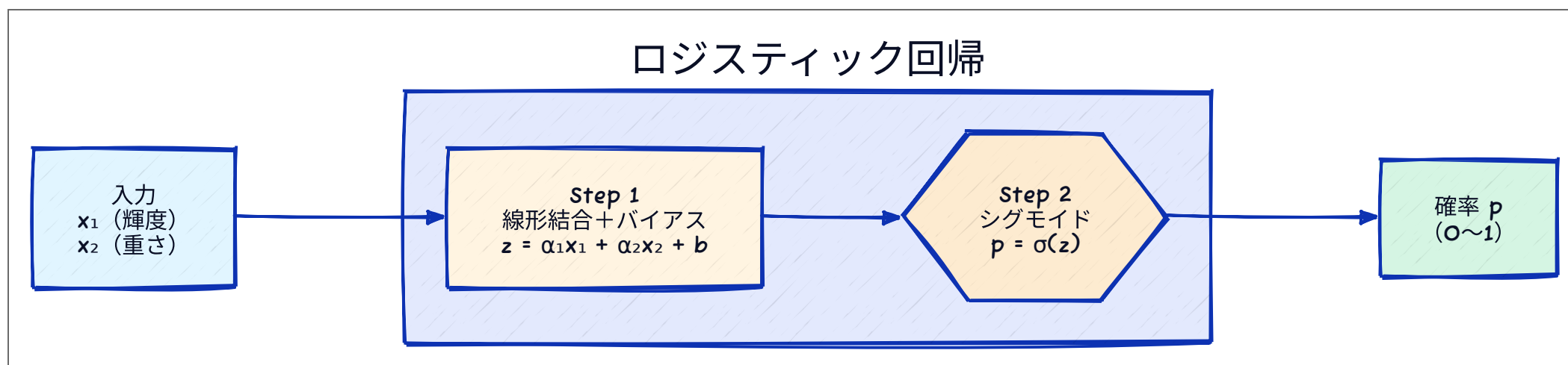
- 前回の線形回帰と **見た目はそっくり**。でも役割は**正反対**
- **回帰直線**：点の**真ん中**を通る — 縦軸の値（重さ）を**予測する**線
- **決定境界**：点の**間**を通る — 2クラスを**分ける**線。予測したい「魚の種類」は軸ではなく**色**で乗っている

5

モデルの構造

6

これがロジスティック回帰 (Logistic Regression)



- 「最適な直線で、上か下かを判別する」機械 — それが**ロジスティック回帰**
- 中身は**2ステップ**：Step 1 が**直線**、Step 2 が**確率** — 1つずつ開ける
- **sklearn** の **LogisticRegression** が中でやっているのもこの計算

① 「回帰」なのに分類？

予測しているのが 0/1 のラベルではなく「Salmon である **確率**」という連続値だから。確率を回帰して、最後にしきい値 0.5 で分類している。

7

Step 1：線形結合+バイアス

$$z = \alpha_1 x_1 + \alpha_2 x_2 + b$$

- x_1, x_2 ：入力（輝度、重さ）
- α_1, α_2 ：**重み (weight)**（どの特徴量をどれだけ重視するか）
- b ：**バイアス (bias)**（全体のオフセット）
- 魚を1匹測って入れると、 z という **1つの数** が出てくる

8

実は、見覚えのある式

- $z = 0$ と置くと：

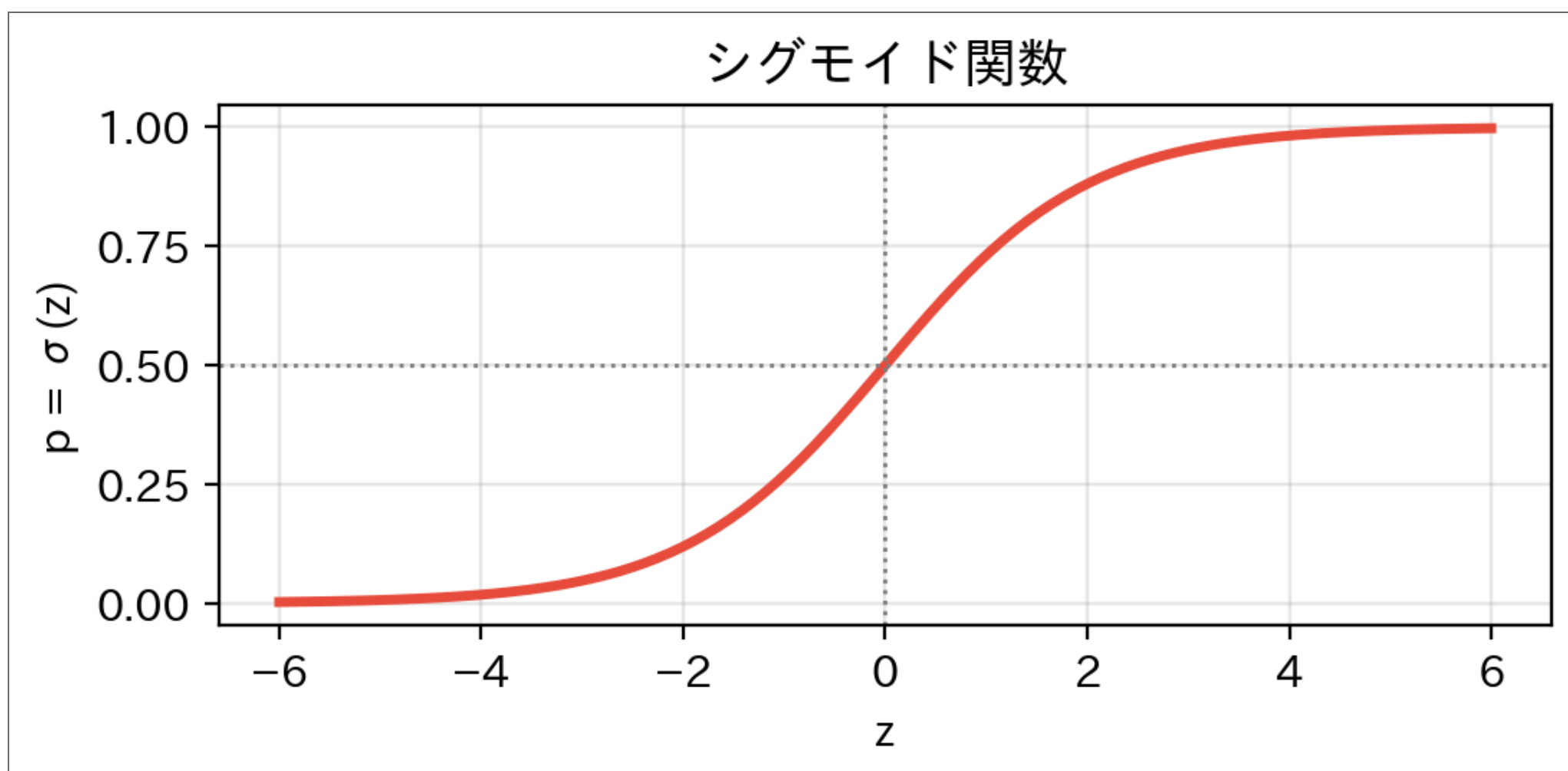
$$\underbrace{\alpha_1}_{a} x_1 + \underbrace{\alpha_2}_{b} x_2 + \underbrace{b}_{c} = 0$$

- 数学でおなじみの**直線の式** $ax + by + c = 0$ そのもの — さっきの斜めの線の正体はこれ（決定境界）
- z の**符号**が「線の上か下か」、**大きさ**が線からの遠さ
- 求めたいのは結局、**この a, b, c （ここでは α_1, α_2, b ）の3つの数** — これを機械に探させる

9

Step 2： z を「確率」にする — シグモイド (Sigmoid)

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$



- 実数全体 $(-\infty, +\infty)$ を **0~1 の範囲** に押し込む
- $p \geq 0.5 \rightarrow$ Salmon、 $p < 0.5 \rightarrow$ Sea bass と判定
- $z = 0$ (=決定境界の上) のとき、ちょうど $p = 0.5$
- 判定だけなら z の符号で足りるが、 p は「**どれくらい自信があるか**」まで持っている — これが後で学習の鍵になる

10

名前の由来 — 何が「ロジスティック」？

- **シグモイド (sigmoid)** は「S字型の」という意味の**総称** — S字カーブの関数はみんなシグモイド
- いま使った $\frac{1}{1 + e^{-z}}$ の正式名は**ロジスティック関数 (logistic function)** — シグモイドの代表選手
- 出自は機械学習ではなく、19世紀の**人口の成長モデル**：最初は勢いよく増え、資源の上限に近づくと頭打ちになる — その成長曲線がこのS字
- ロジスティック関数で確率を出す回帰だから、**ロジスティック回帰**

11

「線形」の正体

12

この式、なんと呼ばれているか

$$z = \alpha_1 x_1 + \alpha_2 x_2 + b$$

- 教科書や解説記事は、これを入力の**線形結合 (linear combination)** と呼ぶ
- ロジスティック回帰は**線形分類器**、前回のは**線形回帰** — この先も「線形」だらけ

📢 クイズ

この「線形」、**直線 (線の形)** のことだと思う？

13

線形結合は線形ではない

- 「線形」は直線のことではない — 実は**この式自体が「線形」の定義を満たしていない**
- その定義 — **線形性 (linearity)** = 「**定数倍と足し算だけ**」:
 $f(cx) = cf(x)$ (斉次性) $f(x_1 + x_2) = f(x_1) + f(x_2)$ (加法性)
- 斉次性 (**homogeneity**): **先に** 定数倍しても **後で** 定数倍しても同じ
- 加法性 (**additivity**): **先に** 足しても **後で** 足しても同じ
- 例: $f(x) = 3x$ は線形。 $f(x) = x^2$ は $f(1+1) = 4 \neq f(1) + f(1) = 2$ で失格
- $f(x) = ax + b$ も失格 — 足してから入れると $+b$ は**1回**、入れてから足すと $+2b$ と**2回** でズレる

14

一発の見破り方: $f(0) = 0$

- 線形なら $f(0) = f(0 \cdot x) = 0 \cdot f(x) = 0$ — **0** を入れたら必ず **0** が出る
- 逆に言えば、 $f(0) \neq 0$ の関数はその場で**線形失格**
- さっき地道に確かめた $y = ax + b$ もこれなら一瞬: $f(0) = b \neq 0 \rightarrow$ **失格**。グラフは**直線**なのに、線形ではない

📢 重要

「グラフが直線」(見た目) と「線形性」(性質) は**別物**。一次関数を「線形」と呼ぶのは、高校までの**見た目**基準。

📖 ノート

$f(0) = 0$ は**失格者を見つける** 検査。 $f(0) = 0$ でも x^2 のように非線形なものはある (合格の証明にはならない)。

15

確かめる： $z = \alpha_1 x_1 + \alpha_2 x_2 + b$

❗ クイズ

「線形結合」を名乗るこの式の、**どこ**が線形の定義を破っているのか？

$f(x_1, x_2) = \alpha_1 x_1 + \alpha_2 x_2 + b$ に、いまの $f(0) = 0$ 検査を当てよ。

- $f(0, 0) = b \neq 0$ — 犯人は b 。中身は「線形結合+おまけの b 」
- さんざん「線形」と名乗りながら、 b が定義を破っている
- 世の中の教科書は、この b を **平気な顔で** 足したまま「線形」と呼び続けている

16

+ b がついていても線形にするトリック

- b を「常に 1 が入っている入力 x_0 への重み α_0 」と読み替える：

$$z = \alpha_0 \underbrace{x_0}_{=1} + \alpha_1 x_1 + \alpha_2 x_2$$

- 入力を $(1, x_1, x_2)$ の3次元と見れば、 $f(0, 0, 0) = 0$ — **正真正銘の線形結合** に戻る
- 統計学ではデータ表に「全部 1 の列」を足す定番の手（切片項）。実装でも広く使われるトリック

i 用語メモ

+ b 込みの形（線形+平行移動）は、文献では**アフィン変換（affine transformation）**と呼んで区別することがある。世の中の説明の多くは + b 込みでも「線形」と書く — 頭の中で $x_0 = 1$ を補えば、その呼び方でも辻褄が合う。

17

線形にこだわる理由

- 「定数倍と足し算だけ」の計算は、**行列のかけ算** にそのまま乗る — 大量のデータを一括で計算できる（NumPy 実装の回で効いてくる）
- 微分しても形が崩れない： $\frac{\partial z}{\partial \alpha_1} = x_1$ — 勾配降下法と相性が最高
- 「線形か、そうでないか」は、**どの道具が使えるか** を分ける最初の仕分けになっている

18

学習 = 境界を動かすこと

19

何を最小化するか：損失関数

- 「いまのパラメータがどれだけダメか」を1つの数字にする

$$L = -\frac{1}{N} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

- 交差エントロピー (cross entropy)** と呼ぶ
- 例：正解 $y = 1$ (Salmon) なのに $p = 0.4$ しか出せない
→ $L = -\log(0.4) \approx 0.92$ (大きい=ダメ)
- 自信を持って正解 ($p = 0.99$) なら $-\log(0.99) \approx 0.01$ (小さい=良い)

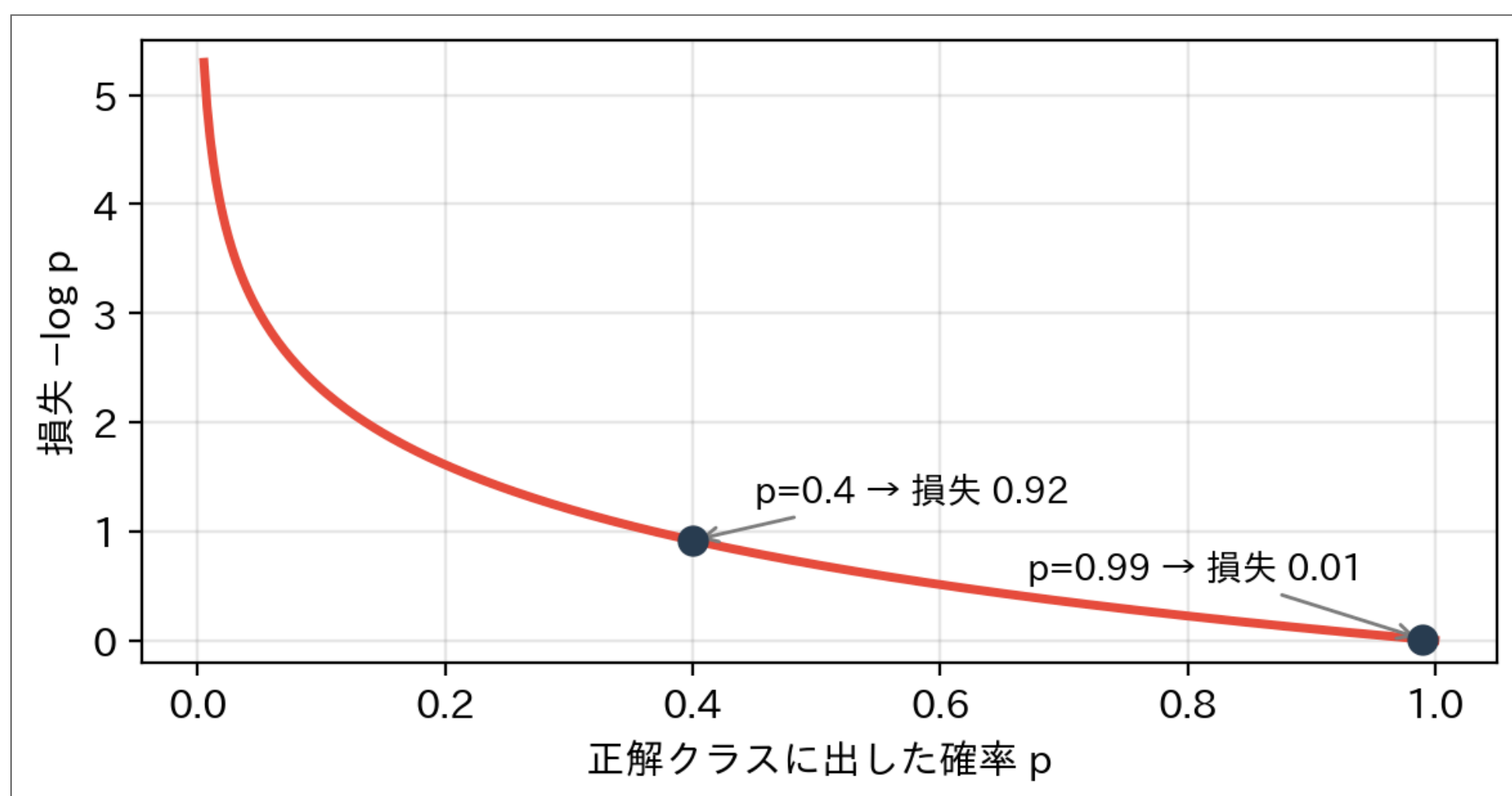
📢 クイズ

知りたいのは正答率。なぜ**正答率そのもの**を最大化しない？

- 正答率はパラメータを少し動かしても**階段状にしか変化しない** → 微分が 0 か未定義
- 勾配降下法が使えない。**滑らかな代理**として交差エントロピーを最小化する

20

外すほど急に効く： $-\log$ の形



- 正解に近い ($p \rightarrow 1$) ほど損失はほぼ 0。外れるほど**急カーブ**で跳ね上がる
- $p \rightarrow 0$ (自信満々の間違い) では青天井 — 機械が最も避けたがるのはこれ
- 正解が $y = 0$ の魚では左右反転して $-\log(1 - p)$ (式の第2項)

21

勾配が驚くほど簡単になる

- 損失 L を z で微分すると、連鎖律 (chain rule) で

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial z} = \left(-\frac{1}{p}\right) \cdot p(1-p) = p-1 \quad (\text{正解 } y=1 \text{ のとき})$$

- 正解が 0 のときは $p-0$ 。つまり 常に

$$\frac{\partial L}{\partial z} = p - y$$

- 各パラメータの更新式 (学習率 η) :

$$\alpha \leftarrow \alpha - \eta(p - y)x, \quad b \leftarrow b - \eta(p - y)$$

① ノート

シグモイドの微分 $p(1-p)$ と交差エントロピーの微分 $-1/p$ が打ち消し合うようにできている。「予測と正解の差 ($p-y$) を、入力の数に応じて配る」だけ。

22

実装：必要な関数は4つだけ

▶ 魚データの定義 (クリックして展開)

Code

```
1 def sigmoid(z):
2     return 1 / (1 + np.exp(-z))
3
4 def forward(X, alpha, b):
5     """線形結合+バイアス → シグモイド"""
6     return sigmoid(X @ alpha + b)
7
8 def loss_fn(y_true, p):
9     """交差エントロピー"""
10    eps = 1e-10
11    p = np.clip(p, eps, 1 - eps)
12    return -np.mean(y_true * np.log(p) + (1 - y_true) * np.log(1 - p))
13
14 def accuracy(y_true, p):
15    return np.mean((p >= 0.5) == y_true)
```

- forward** が予測、**loss_fn** が採点。勾配は $(p-y)x$ なので関数すら要らない

23

魚を1匹ずつ見せて学習する

- 1匹見るたびにパラメータが少し動く

Code

```
1 alpha = np.array([0.0, 0.0]) # 重みはゼロからスタート
2 b = 0.0
3 lr = 0.5 # 学習率
4
5 for i in range(10):
6     xi, yi = X_norm[i], y[i]
7     p_i = sigmoid(xi @ alpha + b) # Forward: 1匹だけ予測
8     delta = p_i - yi # 予測と正解の差
9     alpha = alpha - lr * delta * xi # 勾配降下で更新
10    b = b - lr * delta
11
12    p_all = forward(X_norm, alpha, b)
13    print(f"{i+1}回目 | Loss: {loss_fn(y, p_all):.4f} | 正答率: {accuracy(y, p_all):.1%} "
14          f"| α={[alpha[0]:6.3f], {alpha[1]:6.3f]} | b={b:6.3f}")
```

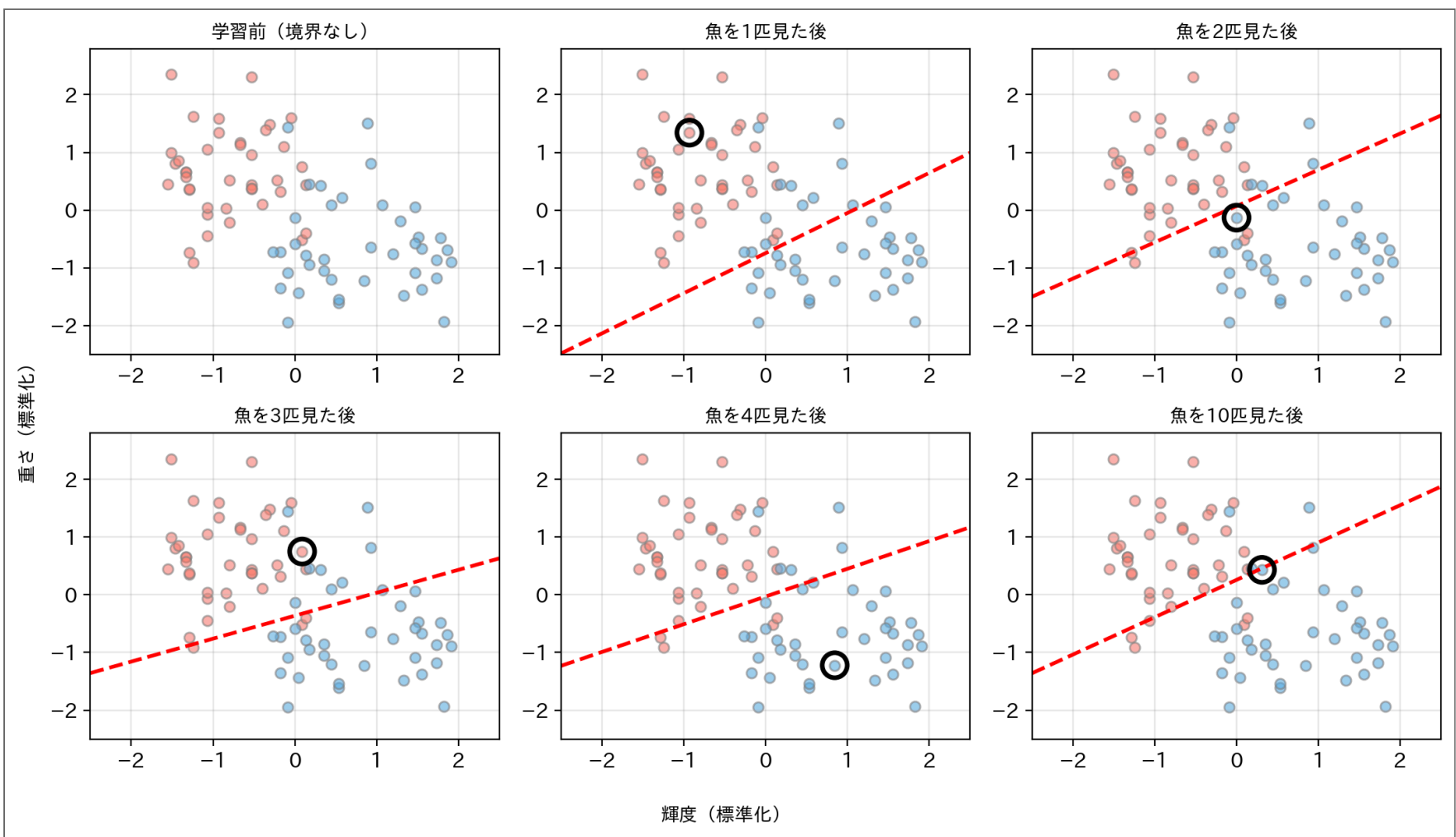
Result

1回目	Loss: 0.5326	正答率: 85.0%	α=[-0.233, 0.335]	b= 0.250
2回目	Loss: 0.5179	正答率: 90.0%	α=[-0.233, 0.371]	b=-0.026
3回目	Loss: 0.4974	正答率: 85.0%	α=[-0.213, 0.537]	b= 0.196
4回目	Loss: 0.4269	正答率: 88.8%	α=[-0.359, 0.748]	b= 0.023
5回目	Loss: 0.4044	正答率: 85.0%	α=[-0.394, 0.914]	b= 0.135
6回目	Loss: 0.3994	正答率: 87.5%	α=[-0.360, 1.054]	b=-0.059
7回目	Loss: 0.3901	正答率: 87.5%	α=[-0.379, 1.137]	b=-0.023
8回目	Loss: 0.3617	正答率: 87.5%	α=[-0.516, 1.206]	b=-0.102
9回目	Loss: 0.3364	正答率: 88.8%	α=[-0.650, 1.272]	b=-0.002
10回目	Loss: 0.3330	正答率: 90.0%	α=[-0.741, 1.146]	b=-0.294

- たった10匹で正答率 90% まで来た

24

境界が動く様子を見る



- ○印＝直前に見せた魚。間違えるたびに境界がその魚の方へ動く
- 縦線しか引けなかった人間と違い、機械は斜めの直線を自由に調整していく

25

学習されたパラメータを読む

- 最終的な重み： $\alpha \approx [-0.74, 1.15]$

パラメータ	値	意味
α_1 (輝度)	負	暗いほど Salmon 寄り
α_2 (重さ)	正	重いほど Salmon 寄り

- 1次元分類・2次元分類の回に **目で見**て発見したことを、機械が数値として再発見した

① ノート

sklearn の **fit()** がやっているのは、本質的にこのループ (の高速・安定版)。 **ブラックボックスが1つ開いた**。

26

Tweak

学習の様子をいじって観察する



- 1匹ずつ学習するコード を書き換えて実行

1 ゆっくり学ばせる `lr = 0.05`

2 80匹全部見せる `for i in range(80):`

💡 ここで観察

- lr = 0.05** : 正答率は 90% で並ぶのに、**Loss は 0.59 vs 0.33** と大差 — パラメータが5分の1しか動いておらず、当ててはいるが**自信がない** (p が 0.5 の近くにいる)。正答率に出ない差を Loss が見ている
- 80匹全部見せると正答率はどこまで上がる？ 途中で**下がる瞬間**が何度もある (間違えた魚に境界が引っ張られる音)

27

まとめ

- ロジスティック回帰 = **線形結合 (+バイアス) → シグモイド**。決定境界は $z = 0$ の直線
- 「線形」は直線の見ただけではなく「**定数倍と足し算だけ**」という性質 — $f(0) \neq 0$ なら線形失格
- $+b$ は厳密には線形破り — 「常に 1 の入力 x_0 」と読み替えれば線形結合に収まる（文献の呼び名：アフィン）
- 学習 = 交差エントロピーを **勾配降下法** で最小化
 - 勾配は $(p - y)x$ — シグモイド + 交差エントロピーの組み合わせの妙
- **fit()** の中身は「予測 → 差を測る → 境界を少し動かす」の繰り返しだった

① 次回予告

この機械、どれだけ学習しても引けるのは**直線だけ**。直線では絶対に分けられないデータが来たらどうする？

28

練習問題

以下の問題を **手計算** と **Pythonコード** の両方で解け。

問題1：線形かどうかを見破る

次の関数が線形性（斉次性・加法性）を満たすか判定せよ。

(a) $f(x) = 5x$ (b) $f(x) = 3x + 2$ (c) $f(x) = x^2$ (d) $f(x_1, x_2) = 2x_1 - x_2$

1. まず $f(0) = 0$ チェックで失格者を探せ
2. 残った関数について、加法性 $f(x_1 + x_2) = f(x_1) + f(x_2)$ が成り立つか確認せよ（破れるなら反例を1つ）
3. コードで「先に足してから入れる」と「入ってから足す」を $x_1 = 1, x_2 = 2$ で計算し、判定と一致することを確認めよ

問題2：ロジスティック回帰を2歩だけ学習させる

2匹だけのミニ世界。標準化済みの特徴量（輝度, 重さ）で、1匹目 $x^{(1)} = (2, 1)$ は Salmon ($y = 1$)、2匹目 $x^{(2)} = (-1, 1)$ は Sea bass ($y = 0$)。初期値 $\alpha = (0, 0)$ 、 $b = 0$ 、学習率 $\eta = 0.5$ 。

1. 1匹目を見せたときの z 、 $p = \sigma(z)$ 、差 $\delta = p - y$ を求めよ
2. 更新式 $\alpha \leftarrow \alpha - \eta \delta x$ 、 $b \leftarrow b - \eta \delta$ で更新せよ
3. 続けて2匹目でもう1歩更新せよ
4. 同じ計算をコードで実行し、手計算と一致することを確認めよ

ヒント： $\sigma(0) = 0.5$ 。実は σ の値はこれしか使わない。

29

問題1の解答例

f(0) チェック

- a. $f(0) = 0$ — 通過
- b. $f(0) = 2 \neq 0$ — その場で線形失格
- c. $f(0) = 0$ — 通過 (ただしまだ合格ではない)
- d. $f(0, 0) = 0$ — 通過

加法性チェック

- a. $f(1 + 2) = 15 = f(1) + f(2) = 5 + 10$ — 成立。斉次性も $f(cx) = 5cx = cf(x)$ で成立 → **線形**
- c. $f(1 + 2) = 9 \neq f(1) + f(2) = 1 + 4 = 5$ — **非線形**。 $f(0) = 0$ を通過しても安心できない
- d. $2(x_1 + x'_1) - (x_2 + x'_2) = (2x_1 - x_2) + (2x'_1 - x'_2)$ — 成立 → **線形**

コードで確認

▶ 解答例を見る

```
Result
|
| (a) 5x      f(0)= 0 | f(1+2)= 15 vs f(1)+f(2)= 15
| (b) 3x + 2  f(0)= 2 | f(1+2)= 11 vs f(1)+f(2)= 13
| (c) x^2     f(0)= 0 | f(1+2)= 9 vs f(1)+f(2)= 5
| (d) 2x1-x2  f(0,0)=0 | f(1+3, 2+1)=5 vs f(1,2)+f(3,1)=5
|
```

- b. は $f(0) = 2$ 、(c) は $9 \neq 5$ でアウト。(a)(d) は一致する — 手計算どおり

問題2の解答例

1匹目 $(x = (2, 1), y = 1)$

$$z = 0 \times 2 + 0 \times 1 + 0 = 0, \quad p = \sigma(0) = 0.5, \quad \delta = 0.5 - 1 = -0.5$$

更新:

$$\alpha \leftarrow (0, 0) - 0.5 \times (-0.5) \times (2, 1) = (0.5, 0.25)$$

$$b \leftarrow 0 + 0.25 = 0.25$$

まだ何も知らないなので $p = 0.5$ (五分五分)。Salmon と言い切れなかったぶん、境界が動いた。

2匹目 $(x = (-1, 1), y = 0)$

$$z = 0.5 \times (-1) + 0.25 \times 1 + 0.25 = 0, \quad p = 0.5, \quad \delta = 0.5$$

偶然にも2匹目は **ちょうど境界の上** ($z = 0$) に乗っていた。

更新:

$$\alpha \leftarrow (0.5, 0.25) - 0.5 \times 0.5 \times (-1, 1) = (0.75, 0)$$

$$b \leftarrow 0.25 - 0.25 = 0$$

コードで確認

▶ 解答例を見る

Result

```
1匹目: z=0.00, p=0.50, δ=-0.50 → α=[0.5  0.25], b=0.25
2匹目: z=0.00, p=0.50, δ=+0.50 → α=[0.75 0.  ], b=0.00
```

手計算どおり、 $\alpha = (0.5, 0.25) \rightarrow (0.75, 0)$ 、 $b = 0.25 \rightarrow 0$ と動く。